ABSTRACT

In the era of ubiquitous digital education, self-directed learners very often face "**analysis paralysis**," inefficient prioritization, and inability to adapt rigid schedules to dynamic real-world constraints. Our team introduces the **Adaptive Study Planner**, an intelligent agentic system for separating the high cognitive load of planning from the act of learning. Unlike traditional study scheduling tools, this system employs a hybrid Artificial Intelligence architecture that constructs, optimizes, and maintains a personalized learning roadmap.

Our project works on a three-layer intelligence pipeline: Perception, Reasoning, and Action. **First**, it leverages LLMs to perceive high-level user goals and decompose these into a semantic Knowledge Graph in the form of a DAG. **Second**, a Rule-Based Reasoning Engine enriches this graph and performs task utility computation over deadlines, difficulty, and user-defined weaknesses. **Third**, the system utilizes an A **Search** Algorithm* to linearize this curriculum into an optimal learning path that it then maps onto scheduled slots using a custom **Constraint Satisfaction Problem (CSP)** solver. The agent exhibits adaptive behavior through the integration of a closed-loop feedback mechanism and is made capable of **XAI** by articulating the logic behind its scheduling decisions. The result is a robust and trustable learning companion that guarantees fulfillment of educational goals with mathematical efficiency and human-centric flexibility.

List of Figures

List of Tables

Abbreviations

Abbreviation	Full Form
AI	Artificial Intelligence
API	Application Programming Interface
BERT	Bidirectional Encoder Representations from Transformers
BFS	Breadth-First Search
CRUD	Create, Read, Update, Delete
CSP	Constraint Satisfaction Problem
DAG	Directed Acyclic Graph

DSA	**Data Structures and Algorithms**
EDA	**Exploratory Data Analysis**
LLM	**Large Language Model**
LSTM	**Long Short-Term Memory**
MOOC	**Massive Open Online Course**
NSGA-II	**Non-dominated Sorting Genetic Algorithm II**
ORM	**Object-Relational Mapping**
PEAS	**Performance, Environment, Actuators,**

	Sensors
RL	**Reinforcement Learning**
SDK	**Software Development Kit**
SVM	**Support Vector Machine**
UI	**User Interface**
XAI	**Explainable Artificial Intelligence**

1. Introduction

1.1 Problem Statement

The move to self-directed learning with online materials in today's educational landscape places the entire burden of curriculum planning on the learner. The democratization of learning through access to information has been counterbalanced by a new critical failure point: Cognitive Overload in Planning.

Learners usually have three compounding problems:

1. **The "Cold Start" Problem:** The inability to break down a high-level goal—e.g., "Learn Deep Learning"—into a structured, granular dependency graph of actionable topics.
2. **The Inflexibility of Static Scheduling:** Conventional solutions, like calendars and to-do lists, are passive containers that have no way to adjust to human variability in energy, gaps in knowledge, or unexpected changes in life.
3. **Optimization Complexity:** Manually balancing the prerequisites, deadlines, difficulty level, and time constraints is a complex constraint satisfaction problem that is beyond the cognitive capacity of most students.

There is a marked lack of systems that actually create a schedule generatively and maintain it intelligently using algorithmic reasoning, rather than merely storing one.

1.2 Objective

The main aim of this work is to develop the **Adaptive Study Planner**, an Agentic Learning Companion, which separates the strategic cognitive load of planning from the tactical act of learning.

Specific Sub-objectives:

- **To Implement Perception:** Use Large Language Models to parse natural language goals into a formal Semantic Knowledge Graph.
- **To Optimize Sequencing:** Apply the A* Search Algorithm in order to determine the most efficient learning path minimizing difficulty spikes while maintaining prerequisites.
- **Scheduling Automation:** Create your own Constraint Satisfaction Problem (CSP) solver that can map variable-duration learning sessions to fixed-time slots in a user's calendar.
- **To Ensure Adaptation:** Implement a closed-loop feedback system wherein the user-defined ratings of Weakness/Confidence dynamically adjust the priority and placement of future tasks.
- **To Provide Transparency:** Incorporate XAI modules in order to explain why certain priorities are made.

1.3 Motivation

The motivation for this work is an observation: "Analysis Paralysis" is the primary barrier to upskilling. As students of Computer Science, we observed that EdTech solutions were either content providers (MOOCs) or passive trackers (Trello/Notion), with a "missing middle": an intelligent agent acting as a dedicated Project Manager for one's education.
By automating the logistics of scheduling, we can minimize the friction when starting and maintaining a study habit. Moreover, the challenge of combining modern Generative AI (content) with Classical Symbolic AI (structure and logic) is a compelling opportunity to build a Hybrid Neuro-Symbolic System.

1.4 Significance

This project extends beyond the simple "CRUD" (Create, Read, Update, Delete) applications into the implementation of actual Agentic Behavior. Its importance lies in:

- **Efficiency in Learning:** The system reduces time lost due to manually navigating through dependencies by automatically sequencing topics with the use of A* Search.
- **Sustainability & Well-being:** The system encourages sustainable learning behavior by incorporating an "Ethics Guard," respect for breaks, capacity limits, and days unavailable.
- **Technical Demonstration:** It is also a proof-of-concept for Neuro-Symbolic AI, which is accomplished by rigid logical structures such as graphs and constraints grounding LLMs to avoid hallucinations and ensure the validity of generated text.

1.5 Challenges

Developing this adaptive agent involved surmounting several key technical hurdles:

- **Dynamic State Space:** Most standard search problems have fixed maps that define the state space. In our case, this is generated dynamically by an LLM. Ensuring the resulting graph is acyclic and valid for A* traversal was one primary challenge.
- **The Bin-Packing Problem:** Scheduling demands the fitting of topics of variable duration, for example, 4 hours, into fragmented time slots such as 1 hour available on Monday and 2 on Tuesday. We needed to develop a custom heuristic for Session Splitting that would maintain sequential integrity.
- **Subjective Weighting:** The algorithm had to weigh conflicting heuristics, like "Get hard topics done early" against "Save hard topics for Peak Energy hours." Tuning cost functions to respect user-defined "Weakness" ratings required careful logic design.

1.6 Novelty Proposed

The Adaptive Study Planner introduces several new approaches, easily distinguishable from standard calendar applications:

- **Hybrid Intelligence Architecture:** We merge the generative creativity of LLMs in curriculum design with the deterministic rigour of CSP in scheduling, so that the plans

generated are comprehensive yet logically executable.

- **Dynamic Session Splitting:** Our custom scheduler plays the role of a constructive heuristic solver, splitting big conceptual topics into manageable sessions based on the user's available time slots.
- **Confidence-Aware Prioritization:** It provides a unique feedback loop wherein the user explicitly rates their confidence or weakness against certain topics. These inputs are utilized by the Reasoning Engine to dynamically boost the priority of weaker areas so they are scheduled in "Peak Focus Hours" rather than being buried.
- **Agentic Explainability:** Unlike black-box optimization tools, our agent provides natural language justifications for its scheduling choices, closing the gap between complex algorithms and user trust.

2. Literature Review

Educational technology has emerged to shift away from the dead learning type of teaching to dynamic and learning assistance that is personalized. Initial studies by Tiwari and Singh [1] show that the major weakness of conventional scheduling tools lies in their inability to be active instruments of optimization within human constraints, but as passive storage repositories of dates. They claim that good self-directed learning necessitates the existence of an Intelligent Scheduler that can reason about time and about user behavior, which our project also fulfills.

Nonetheless, creation of the learning content is still a challenge. Li et al. [2] suggested the application of Large Language Models (LLMs) in order to customize learning paths, although they reported that LLMs find it difficult to be consistent over the long term. Abu-Rasheed et al. [3] resolved this by suggesting a hybrid solution: to use LLMs to hallucinate original ideas and refine them in structured Knowledge Graphs. This is our validation step and it is core to our Perception Layer, as we make sure that the state space we have created is topologically sound, before any planning takes place.

After the definition of the curriculum, the resource allocation issue arises. Zhang [4] authoritatively classifies university timetabling as a Constraint Satisfaction Problem (CSP), and the author provides evidence that a heuristic logic tends to be better at solving a problem presented as "Soft Constraints" such as preferences, which was one of the principles on which we based our custom scheduler. Lastly, the theoretical framework of our overall architecture is given by Colelough and Regli [5] along with other authors. They refer to Neuro-Symbolic AI as a fusion of Neural networks (perception) and Symbolic logic. Our Adaptive Study Planner is an actual application of this architecture, which synthesises the generative flexibility of Li with the mathematical rigour of Zhang.

2.1 Comparative Study of Related works

Reference	Methodology / Focus	Applicability to Adaptive Study Planner
[1] Tiwari & Singh (2024)	AI-Powered Smart Planners	Describes the problem statement: Static tools do not adjust to the user behavior and they are prone to burnout.
[2] Li et al.	LLMs in Learning Paths	Authenticates our use of A* Search on a graph instead of

		using LLM generation alone in sequencing.
[3] Abu-Rasheed et al. (2025)	Knowledge Graph Completion	Provides insight into our design of a Perception Layer: we extract JSON out of LLMs to create a rigorous NetworkX graph.
[4] Zhang (2024)	CSP in Timetabling	Arguments Why we should model our scheduler as a CSP with Hard (Overlap) and Soft (Peak Hours) constraints.
[5] Colelough and Regli (2024)	Neuro-Symbolic AI	Classes our system architecture (Neural Perception + Symbolic Reasoning) as the current state of the art in having reliable agents.

2.2 Key Technical Dependencies

The deployment is based on the following standard libraries and frameworks:

- **NetworkX:** Python library of network structure, editing and analysis of the structure, dynamics and dynamics of complex networks. Applied in this project to model the State Space (Curriculum Graph) and execute the A* Search Algorithm.
- **Python-Constraint:** This is a library providing solvers to Constraint Satisfaction Problems (CSPs) over finite domains. Although our scheduler is a home-written heuristic, it is designed using the concepts of variables/domain in this library.
- **Gemini SDK:** The visual interface of Google Gemini models. It is the Perception Engine and does the unstructured-to-structured data transformation (Natural Language to JSON) and XAI (Explainability) layer.
- **Streamlit:** An open-source Python library used to create interactive web applications, which support both machine learning and data science. It gives the User Interface and maintains persistence of state of a Session.

3. Exploratory Data Analysis

3.1 Dataset and Data Acquisition

The Adaptive Study Planner is based on a Dynamic Generative Data Pipeline, unlike the more traditional Machine Learning projects, which use static and pre-labeled datasets (ex: Kaggle repositories). The system is not downloaded, it is constructed in real-time according to the intent of the user into a personalized dataset.

3.1.1 The Generative Pipeline

The largest source of data is a Large Language Model (Google Gemini 2.5 Flash), which serves as a synthetic data generator. The flow of acquisitions is as follows:

- **Input:** The user enters a high-level intent query (e.g., "Learn Data Structures").
- **Prompt Engineering:** This intention is being enclosed in a rigid system implementing a JSON schema, which is asking particular attributes: Title, Estimated Hours, Difficulty (0-1 scale), Category and Subtopics.
- **Validation Pipeline:** The raw string output of the LLM is run through a Regex Parser (defensive programming) and a Pydantic model (TopicNode) to assert the integrity of data types.
- **Persistence:** Persistence is contained in an SQLite Database through SQLAlchemy ORM.

3.1.2 Data Schema (The "Dataset")

The Directed Acyclic Graph (DAG) represents the resulting dataset of any particular user. Our database (TopicNode) is a row based database with the following characteristics of each data point:

- **ID (Primary Key):** Unique identifier.
- **Dependencies (Edges):** List of Parent IDs (Prerequisites).
- **Quantitative Characteristics:** Estimated Duration (Minutes), Difficulty Scalar (0.0 - 1.0), Priority Score.
- **Qualitative Features:** Subtopic Lists, Theory/Practice (Category Tags).

This dynamic system enables the system to possess a dataset of practically infinite size, and any domain whatsoever the user requests, as opposed to a hard-coded syllabus.

3.2 Feature Engineering & Exploratory Data Analysis (EDA)

Although the data is generated in real time, the system does real-time analysis of the graph properties to feed the AI reasoning engines.

3.2.1 State Space (Graph Topology) Analysis

The system displays the data in the form of a network. With the help of streamlit-agraph, Plotly, we examine the topological sort of the created curriculum:

- **Dependency Density:** We consider the out-degree of the nodes. The nodes that have a large out-degree (blocking a large number of future topics) are considered as Bottleneck Nodes.
- **Critical Path Analysis:** A* Search algorithm implicitly calculates the cost path to the leaf nodes, the effect of this is determining the "Longest Path" in the data that determines the minimum course time.

3.2.2 Visualization

The project uses three different layers of visualization to interpret the underlying data:

- **Force-Directed Graph:** This is a physics-based visualization tool that groups similar topics and lets the user see clusters of heavy study loads (e.g. a heavy cluster of Math topics).
- **Categorical Distribution:** The bar chart analysis of the distribution of topics in categories (Theory vs. Practice). This warns the user against a generated plan being too theoretical.
- **Difficulty Heatmap:** Nodes are color-coded with the feature of difficulty (0.0-1.0), which will give a visual heatmap of the position of the most difficult weeks in the schedule.

3.2.3 Feature Engineering (Priority Synthesis)

In order to facilitate smart scheduling, raw data features are converted to engineered features that propagate the Constraint Satisfaction Problem (CSP) logic. This reasoning is contained in the PriorityCalculator.

Engineered Feature 1: Composite Priority Score (P_{score}) We do not use one attribute to make sorting. We design a composite attribute P_{score} Priority Queue, used to prioritize tasks in a queue:

$$P_{score} = (w_d \cdot \text{Difficulty}) + (w_u \cdot \text{Urgency}) + (w_w \cdot \text{Weakness}) + (w_p \cdot \text{PrereqCount})$$

- **Urgency Transformation:** The raw deadline date is converted into a non-linear urgency score. As $\Delta days \to 0$, The score is exponentially decreasing which makes the scheduler focus on immediate tasks.
- **Weakness Injection:** User confidence (1-5) scores are scaled to the inverse and an inverted scale (to form a Weakness feature), which then allows the system to weigh-up a topic more heavily where the user is not confident.

Created Capability 2: Fragmentation of Sessions Estimated Duration (e.g., 4 hours) is provided in the raw data. A 4-hour block cannot be scheduled in any real fragmented calendar. To create a Session_Chunks feature, we break down the scalar duration into discrete vectors (i.e. [60min, 60min, 60min, 60min]) that have metadata such as Part Index.

3.3 Agent Specification (PEAS Framework)

We rationalize the Adaptive Study Planner as an agent before studying the state space that is

generated. We consider this a Utility Goal-Based Agent, because it does not only attempt to achieve a goal (learns everything), but it tries to maximize an internal utility function (attempting to optimize time, difficulty balance, user preference).

Formal PEAS (Performance, Environment, Actuators, Sensors) description is the following:

Component	**Project Contextual Description**
Performance Measure	The agent maximizes: **Coverage:** Making the most of the amount of topics to be covered within the deadline. **Constraint Compliance:** No hard constraint (dependency and availability) violations. **User Wellness:** Reducing the risk of burnout (break ratios and study limits). **Schedule Utility:** Pushing "Weak" topics into "Peak Focus Hours."
Environment	**Dynamic, Partially Observable, Discrete:** The environment is comprised of the User Calendar (Time Slots), the Curriculum Graph (Knowledge Dependencies), and External Constraints (Deadlines). It is dynamic as the availability of the user and progress change with time.
Actuators	**Virtual & Interface Actions:** **Generate Schedule:** Finds connections between nodes of the topic and time in the calendar. **Re-prioritize:** Updating Priority Score of nodes in the State Graph. **Notice/Alarm:** Impossible Schedules/Burnout Risk warning.

	Explain: Natural Language Justifications using the LLM.
Sensors	**Digital Perception & Direct Input:** **UI Widgets:** Streamlit form to fill in Goal, Deadline, Availability Grid, and Weakness Ratings. **System Clock:** Tracking the set deadlines and the current date. **Loop:** “Difficulty Sliders” feel the performance of the user after the completion of the task. **LLM Parser:** The semantic structure of natural language can be perceived based on prompts.

4. Methodology

4.1 Problem Statement

4.1.1 Define the Problem

A well-acknowledged failure mode that self-directed learners usually experience is termed "planning fallacy." Although humans are good at picking a goal, for instance, "I want to learn AI," they lack the computational ability to solve the Dynamic Constraint Satisfaction Problem of scheduling. Learners cannot decompose such abstract goals into granular dependencies; they cannot consider prerequisites, trying to learn advanced topics without learning the basics first; and they cannot dynamically reshuffle their schedules when some unexpected events happen. The result is rigid, static plans that break under real-world pressure and lead to the abandonment of the learning goal.

4.1.2 Relevance to AI and Real-World Applications

This problem goes beyond simple data processing; it involves intelligence. A standard algorithmic solution, such as using a calendar app, stores an entry but cannot perform any form of reasoning over it. An intelligent approach is necessary since the system must:

- Perceive unstructured intent ("Learn Python").
- Reason about the trade-offs: Difficulty vs. Time vs. Priority.
- Plan a sequence through a directed graph.
- Adapt to new information - User feedback.

4.2 State Space Search

We model the curriculum as a pathfinding problem through a semantic graph to determine the optimal learning sequence. Unlike the standard To-Do list, we treat the sequence of topics as a search for the path of least "Cognitive Resistance" from ignorance to mastery.

4.2.1 State Space Definition

We define the learning environment as a search problem P with the following components:

- States (S A state represents the user's current knowledge snapshot. More formally, a state s is the set of all topics the user has completed.
  $s \subseteq \text{AllTopics}$
- **Initial State (S_0):** An empty set $\emptyset$ Representing the beginning of the learning journey.
- **Goal State (G):** The state where the "Completed" set equals the entire set of topics generated by the curriculum.
- Actions (A): An action a_t is defined as "Studying Topic t." An action is valid only if the topological constraints of the graph are met. In particular, a topic can be selected only if all its prerequisites exist in the current state:

$$\text{Valid}(a_t, s) \iff \text{Prerequisites}(t) \subseteq s$$

- **Transition:** The completion of a valid topic moves the system to a new state $s_{new} = s_{old} \cup \{t\}$. Thus, it effectively adds new edges to the graph.

4.2.2 Search Strategy: The A* Algorithm

We chose the A* search algorithm for sequencing of the graph. While algorithms such as Breadth-First Search (BFS) or Topological Sort can sequence a dependency graph, they inherently treat all nodes as equals. In any real-world learning scenario, topics will differ significantly in terms of Duration, Difficulty, and Strategic Importance. A* enables us to optimize the sequence based on these factors using the evaluation function.

$$f(n) = g(n) + h(n)$$

4.2.3 The Cost Function $g(n)$

The "intelligence" of our planner lies in how it calculates the cost. We do not simply measure time, we measure Cognitive Load. The cost $C(t)$ of including a certain subject in the agenda is calculated as the weighted sum:

$$C(t) = \text{Time}_{est} + (\alpha \cdot \text{Difficulty}) - (\beta \cdot \text{Priority})$$

This equation drives intelligent behavior via three mechanisms:

- **Time:** Base cost. Longer topics are expensive, which encourages the scheduling of quick-win prerequisites first in order to maintain momentum.
- **Difficulty Penalty (α):** We add a penalty for high difficulty (0.0–1.0). That way, if possible, the search will try to find easier paths to the same milestone in order to smoothen the learning curve.
- **Priority Reward (β):** We subtract cost for High Priority. If the Reasoning Engine marks a topic as "Critical" (for example, because of an upcoming deadline), we drastically lower its cost. This makes the node "mathematically attractive" to the A* algorithm, and it is forced to the front of the queue.

4.2.4 Heuristic Function $h(n)$

A heuristic is used to efficiently guide the search toward the goal without exploring every permutation.

$$h(n) = \sum_{t \in \text{Unvisited}} \text{EstimatedTime}(t)$$

Admissibility & Optimality:
For A* to be optimal, the heuristic should not overestimate the cost. As our heuristic includes only the sum of Raw Time (ignoring the additional penalties for Difficulty included in the actual cost), it is strictly admissible ($h(n) \leq \text{TrueCost}$). This will ensure the agent can always find the

most efficient sequence possible.

4.2.5 Implementation

This logic is implemented using a Priority Queue (Min-Heap) in the file backend/search/a_star_planner.py.

- **Robustness:** The implementation includes a fallback mechanism. In case the LLM generates a graph with cyclical dependencies—like, for instance, $A \to B \to A$—infinite loop capability detection, gracefully degrading to the standard fallback sort, ensures that the user will always get a valid plan.
- **Data Structure:** The use of frozenset to represent states guarantees hashability and efficiency for loop detection.

4.3 Knowledge Representation

However, to make intelligent reasoning possible, the system has to do more than merely store data: It has to represent the relationships, attributes, and logic of the learning domain in a machine-understandable format. We use an architecture of **Hybrid Knowledge Representation**, which merges three different AI paradigms: Semantic Networks, Frame-Based representation, and Procedural Rules.

4.3.1 Representation Techniques & Implementation

1. Semantic Network (The Curriculum Graph)
We model the learning curriculum as a Semantic Network, precisely a Directed Acyclic Graph or DAG. This will be the primary structure used for the State Space Search.

- **Implementation:** defined in backend/search/state_graph.py using the NetworkX library.
- **Structure:**
  - Nodes (V): Represent conceptual entities, or Topics. Each node carries semantic attributes: Difficulty scalar ranging from 0 to 1, Category-Nominal: "Theory"/"Practice", and Estimated Duration.
  - Edges (E): Represent the semantic relationship Is_Prerequisite_Of(A, B). An edge $A \to B$ implies knowledge of A is a strict dependency for B.
  - Sub-symbolic Data: Nodes also contain subtopics, a list of granular concepts used for session splitting, derived from the LLM's output.

2. Frame-Based Representation (The User Model)
We use a Frame-Based System, whereby the user's constraints and preferences are modeled. In the proposed system, a "Frame" allows us to arrange knowledge about a particular entity using "Slots" and "Fillers."

- **Implementation:** It is defined in backend/models/user_model.py using the Pydantic library, which enforces schema validity.
- **The Frame (User):**
  - Constraint Slots: unavailable_slots (Matrix of blocked times), deadline (Timestamp).

- Preference Slots: learning_style = Visual/Text, peak_focus_hours.
- Dynamic Slots: topic_weakness - a dictionary that maps Topic IDs to user confidence scores.
  This representation serves as the "Context" to the reasoning engine so that decisions of the agent remain firmly grounded in reality particular to the user.

3. Production Rules (Procedural Knowledge)
Logical heuristics related to "How to study effectively" are represented as Procedural Knowledge in terms of IF-THEN Production Rules.

- **Implementation:** Available in backend/reasoner.py.
- **Logic Flow:** The Inference Engine cycles through the Graph and User Frame to arrive at new facts known as Priority Scores.
- **Example Rule:** IF (Deadline < 7 Days), THEN Boost Priority by 50%
- **Example Rule:** IF Topic is Difficult AND User Preference is "Start Easy" THEN Reduce Priority by 20%

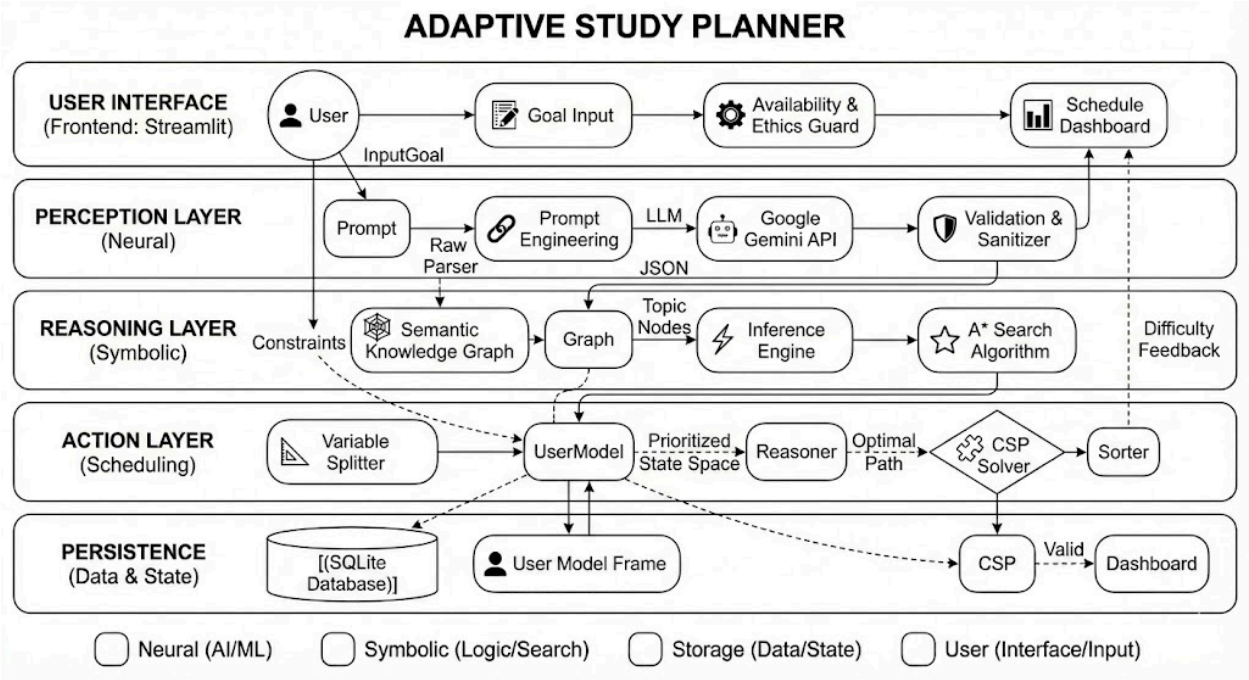
4.3.2 Appropriateness and Justification

We have chosen this hybrid approach over other methods, such as pure Logic Programming or Vector Embeddings, for three reasons:

- **Hierarchical Integrity:** Learning is inherently hierarchical. A Graph is the only mathematical structure that strictly enforces dependency integrity, preventing for example the scheduling of "Advanced Calculus" before "Algebra", which simple databases cannot do.
- **Type Safety & Validation:** Frames allow rigorous data type validation. Since our input is generated by a Generative AI (LLM) that can hallucinate, mapping the output to a strict Frame ensures invalid data (e.g., negative durations or circular dates) is rejected before it ever reaches the planner.
- **Explainability:** Rule-Based Systems are deterministic and transparent. When the agent schedules a task, we can trace the decision back to a specific rule firing, that is, "This was scheduled because Rule 3 (Deadline) triggered". This would be impossible with opaque "Black Box" neural network planners.

4.4 Intelligent System Design

The Adaptive Study Planner is designed as a Model-Based, Utility-Driven Agent. Unlike the simple reflex agents, which directly respond to immediate inputs, this system maintains an internal model of the world—the Knowledge Graph and User Frame—and acts with the intent to maximize a utility function, namely learning efficiency.



4.4.1 System Architecture

We use the Neuro-Symbolic Architecture, which fuses Large Language Models' generative capabilities (Neural) with the reliability of traditional Search and Constraints (Symbolic). The flow of data follows the well-known Sense-Think-Act cycle:

- **Perception:** The system accepts high-level goals and constraints from the user. It enriches this input through a "Generative Sensor" for the purpose of abstract goal decomposition into concrete data structures.
- **Reasoning:** The core intelligence engine processes the input from above. This enlivens the state space, utilizing logical inference rules, calculates path costs using A*, and sequences the learning journey.
- **Act:** Planner converts the abstract sequence to a concrete schedule; the implementation involves a Constraint Satisfaction.
- **Adaptation-Learn:** Closing the loop, the actual user performance is captured—a measure of difficulty ratings—influencing future reasoning cycles by updates to its internal User Model.

4.4.2 Components and Functionalities

The system is modularized into four distinct computational layers, namely:

1. The Perception Gateway

This module addresses the "Cold Start" problem. It does not perform any planning; rather, it acts like a structured data generator.

- **Function:** Takes a natural language string as input, such as "Learn System Design", and returns a JSON Schema with strict fields: Title, Estimated_Duration, Prerequisites, and

Subtopics.

- **Defense:** Includes a Validation Layer (Regex & Pydantic) to clean the LLM outputs. Precludes "Hallucination" of non-existing dates and negative durations from flowing into the planner downstream.

2. The Inference Engine

This module represents the agent's "Contextual Awareness." It is situated between the static graph and the active planner.

- **Purpose:** It performs Forward Chaining Inference. It reads the static node attributes and the dynamic UserModel.
- It attaches dynamic metadata to graph nodes, like Critical_Flag for upcoming deadlines or Review_Needed for spaced repetition; it calculates the final priority score used by the search engine.

3. The Optimization Engine

This layer sequences the material.

- **Function:** This class implements the A* Search algorithm. It linearises the DAG provided by the Perception layer.
- **Role:** It is the "Strategic" planner, deciding on the What and the Order of study, not being concerned with time-slots.

4. The Scheduling Engine

This layer assigns the material to time.

- **Function:** A Constructive Heuristic Solver. It iterates through the ordered list from the Optimization Engine and tries to "bin-pack" the topic sessions into the user's calendar.
- **Role:** It acts as the "Tactical" planner, which deals with the physics of time: duration, breaks, and blocked days.

4.4.3 Novelties

This design introduces three specific technical innovations:

- **Dynamic Session Splitting:** Standard planners are used to slot whole tasks. Our system handles granular decomposition: if a topic requires 4 hours but the user has only two 2-hour blocks on different days, then the logic in scheduler.py intelligently fragments the topic into "Part 1" and "Part 2," creates a sequential constraint between them, and schedules them separately.
- **Confidence-Driven Priority Adaptation:** We made a specific feedback loop in 4_Progress.py, in which the user's self-reported Weakness Confidence Rating will directly alter the weight edges in the A* graph to ensure the automatic increase of urgency of concepts at which the user struggles and force them into "Peak Focus" timeslots.
- **Explainable Logic Layer (XAI):** The system offers Traceability. Because the architecture separates the Rule Logic, reasoner.py, from the Generation, llm_helper.py, the agent is able to explain explicitly its decisions, such as "I prioritized X because of Rule 3: Deadline

Urgency", rather than giving opaque, black-box recommendations.

4.5 Constraint Satisfaction Problem (CSP)

The A* Search identifies the best calendar of topics, but does not take into consideration the physics of time of a calendar (e.g. Monday has 3 hours free, Tuesday has 0). We use the assignment of these topics to each time slot as a Constraint Satisfaction Problem.

4.5.1 Formal Definition $\langle X, D, C \rangle$

The scheduling problem is a triple which we define as:

Variables (X)
The variables are the particular Study Sessions that are necessary to address the curriculum. The sole problem with our field is that the variables are not fixed; they keep on changing.
A Topic T_i with duration $Dur(T_i)$ is decomposed into k sessions $S_{i,1}, S_{i,2}, ..., S_{i,k}$ depending on the session desired by the user (lesson) $L_{session}$.
$X = \{S_{1,1}, ..., S_{n,k}\}$ denotes the collection of topic fragments that are to be planned.
Domains (D)
The domain is the finite list of valid time slot within the scheduling horizon (usually one week).
Let $Days = \{Mon, Tue, ..., Sun\}$.
Let $Slots = \{06-09, 09-12, 12-15, 15-18, 18-21, 21-24\}$.
The raw domain is $D_{raw} = Days \times Slots$.

- Pruning: We define the domain explicit: We simplify the domain by deleting user-formulated blocked times:
  $D_{valid} = D_{raw} \setminus \{(d, t) \mid d \in \text{UnavailableDays} \vee t \in \text{UnavailableSlots}_d\}$

Constraints (C)
Our Hard Constraints (must be satisfied) and Soft Constraints (optimization targets).

- **Hard Constraints:**
  - Unary Resource Constraint: A session $S_{i,j}$ can not occupy a slot whose available capacity is lower than the duration of the session.
    $\sum_{S \in Slot_k} \text{Duration}(S) + \text{Breaks} \leq \text{Capacity}_k$
  - Sequential Precedence (Temporal Ordering): In the case of a particular topic T_i, Part j must strictly follow Part $j-1$.
    $\text{EndTime}(S_{i,j-1}) < \text{StartTime}(S_{i,j})$
  - Daily Load Limit: The cumulative amount of time that is allocated to Day d should not be more than the user prescribed limit.
    $\sum \text{Duration}(S \in \text{Day}_d) \leq \text{UserDailyLimit}$
- **Soft Constraints (Preferences):**

- **Energy matching:** If $\text{Difficulty}(T_i) > 0.7$, prefer $D \in \text{PeakHours}$.
- **Weakness Prioritization:** If $\text{Weakness}(T_i) > 0.8$ I like to work earlier in the week.

4.5.2 Solution Strategy

Generic CSP solvers (such as backtracking search) are used to solve valid assignments, but have difficulties in the variable packing that is needed here (packing 45-minute chunks into 3-hour blocks).
A variant of Greedy Local Search (Constructive Heuristic Search) was used in our backend/csp/scheduler.py.
The Algorithm:

1. Decomposition: The algorithm transforms the list of Topics of the A* planner into a priority-sorted queue of particular Session Variables. The heuristic used to sort is a composite one:
   $H(s) = (w_1 \cdot \text{Priority}) + (w_2 \cdot \text{Weakness}) + (w_3 \cdot \text{Difficulty})$
2. **Greedy Assignment:** Given a specific unassigned Session S we move through the ranked Domain D_{valid} (ordered chronologically).
3. Evaluation Function: To each of the possible Slots we assign a score of suitability:
   $\text{Score}(S, \text{Slot}) = \text{RemainingCapacity} + \Delta(\text{IsPeak}, \text{Difficulty}) - \text{DeadlinePenalty}$
4. **Assignment:** The session is dedicated to the slot with the highest positive score which meets Hard Constraints. In case no slot is valid (Constraint Violation), the session is put on a dropped queue (emulating a No Solution state of that particular variable) instead of crashing the system.

This will ensure that the material that has the greatest priority will be run in the optimum time slots first and this will ensure strict adherence to the learning strategy of the user.

4.6 Other AI Techniques

While core reasoning and planning rely on deterministic Symbolic AI, the system uses recent advances in Generative AI to deal with unstructured data and user interaction.

4.6.1 Large Language Models as Perception Layers

We integrated the Google Gemini model (gemini-2.5-flash) to function as the agent's primary perception and generation engine.

- **Solution: "Cold Start"** - One of the biggest obstacles in symbolic planning systems is to populate the knowledge base. It is cumbersome for the users to manually define more than 50 nodes with prerequisites. We use the LLM to auto-construct the state space.
- **Implementation:** The module backend/utils/llm_helper.py uses techniques in Prompt Engineering like Chain-of-Thought and JSON Constraint prompting. We've told the model what to do: "Act as a curriculum designer. Return strictly valid JSON array. Each object

must have {id, title, prereqs}."

- **Defensive AI:** The system implements a Sanitization Pipeline to mitigate the stochastic nature of LLMs (hallucinations). It parses the raw output via Regex to extract JSON objects that are then validated against a Pydantic schema. If the LLM returns a cyclic dependency (A requires B, B requires A), it will be detected and rejected by the Symbolic layer.

4.6.2 Explainable AI (XAI) Module

To increase trust in the agent's decisions, we utilize the LLM as a Natural Language Translator for the system's internal logic logs.

- **Function:** The generation of a schedule creates raw artifacts - for example, Topic A was placed in Slot 3, Score=120. These values cannot be interpreted by a regular user.
- **Process:** We feed a summarised logic log into the LLM with the prompt: "Explain why the scheduler prioritized Topic A over Topic B based on these constraint tags."
- **Relevance:** This fulfills the "Explainability" need of an intelligent agent, where opaque algorithmic calculations are converted to human-readable justification ("I scheduled this early because you marked it as a Weakness.").

4.6.3 Integration & Relevance Neuro-Symbolic Architecture

The system implements a Neuro-Symbolic integration approach:

- **Neural:** Handles high-level pattern matching, content generation, and semantic understanding of the "Learning Goal."
- **Symbolic (A*/CSP):** Carries out the logical verification, sequencing, and strict constraint checking.

This provides the system with the creativity to come up with a roadmap on any topic, from "Guitar" to "Quantum Physics," while retaining the reliability of a mathematical solver and avoiding common LLM mistakes like scheduling two events at the same time.

4.7 Bonus Points

4.7.1 Originality

- **Granular Session Splitting:** the logic inside scheduler.py responsible for breaking up scalar duration attributes into sequential session objects while maintaining "Part X of Y" ordering is a unique heuristic implementation that was developed for this project to solve the "Bin Packing" problem in student schedules.
- **Hybrid Feedback Loop:** Explicit user confidence ratings - Logic - are combined with LLM-generated content - Generative - to create a feedback loop that heals the curriculum over time.

4.7.2 Ethical Considerations & Safety

We designed specific mechanisms aimed at aligning the agent with human values, including

well-being, autonomy, and transparency.

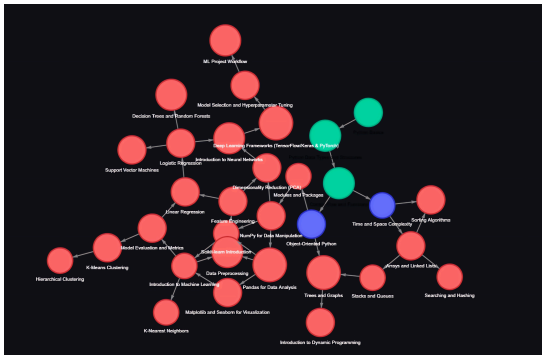
- **Cognitive Ergonomics & Burnout Prevention:** Optimization algorithms typically maximize throughput, meaning do as much as possible. We inverted this to maximize sustainability. The EthicsGuard class in our backend strictly enforces Human-Centric Constraints:
    - **Break Ratios:** The scheduler is mathematically unable to book back-to-back blocks without inserting minimum recovery intervals - 10 mins per hour.
    - **Load Capping:** If a user attempts to configure a schedule with >8 Hours of intensity, the system flags this as a health risk, prioritizing user physiology over algorithmic efficiency.
- **User Autonomy (Human-in-the-Loop):** To avoid "Algorithmic Enslavement," the system allows granular definition of unavailable_slots. These user-defined constraints are treated by the CSP solver as Hard Constraints that override any optimality calculation. The AI serves the user's life pattern rather than forcing the user to conform to the AI's optimization curve.
- **Mitigating Bias via Explainability:** Automated decision systems often have inherent "Black Box" bias. By integrating an XAI module, we make the system accountable to explain the logic behind its prioritization-for example, "I scheduled 'Math' at 9 AM because you rated your confidence low." This kind of transparency helps the user confirm that the system indeed implements valid logic and is not acting based on arbitrary or biased data patterns.

5. Results

5.1 Perception and Knowledge Construction

Objective: To validate whether the LLM-driven perception layer correctly interprets high-level goals into granular dependent topics.
Observation (Figure 6.1):

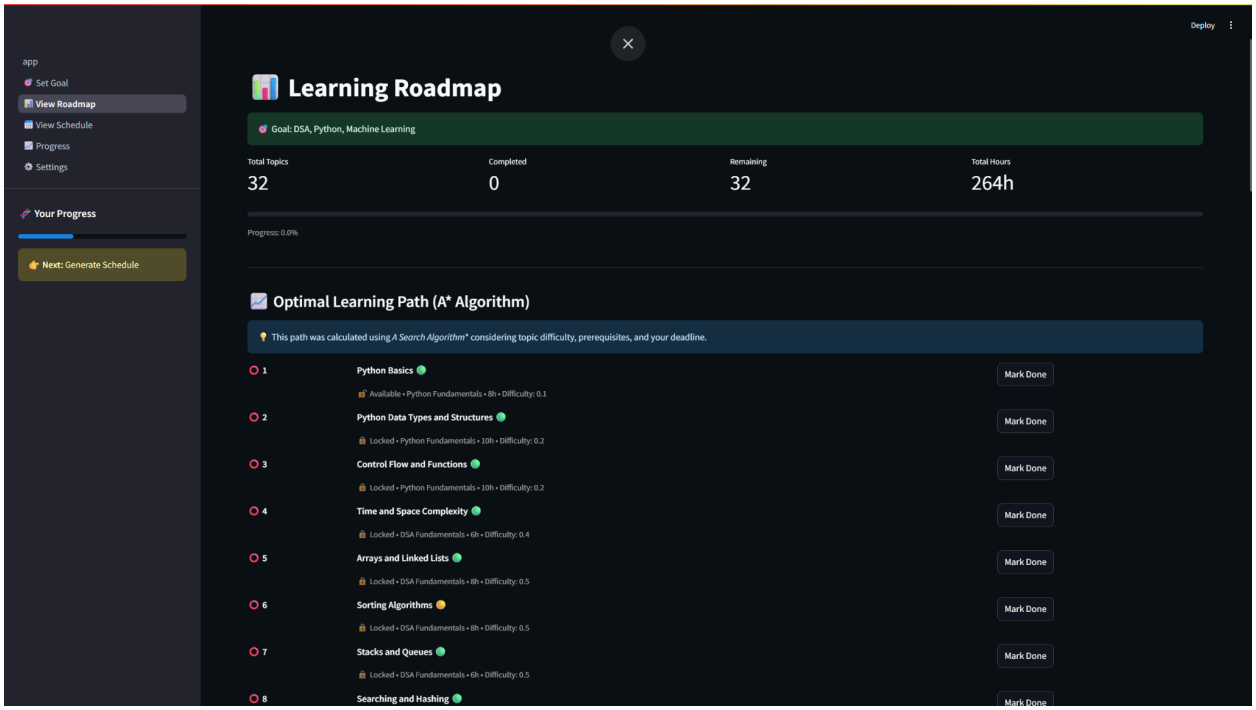
- **User input:** multi-domain objective, "DSA, Python, Machine Learning". The system generated one comprehensive 32-node curriculum, as shown in the roadmap dashboard.
- **Semantic Correctness:** Knowledge Graph clustered similar concepts as shown in Figure 6.2. For instance, it managed to capture that "Python Basics" should strictly appear before "Deep Learning Frameworks", which shows semantic awareness.
- **State Visualization:** The generated graph visualizes the State Space. The system color-coded nodes based on validity:
  - Blue/Green: Available/Completed States.
  - Red: Locked States (Prerequisites unmet).
  - This confirms that the graph generation logic correctly enforced topological dependency rules.



5.2 Learning Path Optimization A* Search

Objective: To prove the A* algorithm gives a logical linear sequence from the non-linear graph.
Observation (Figure 6.3):
The "Optimal Learning Path" list explicitly orders topics based on the cost function $f(n) = g(n) + h(n)$.

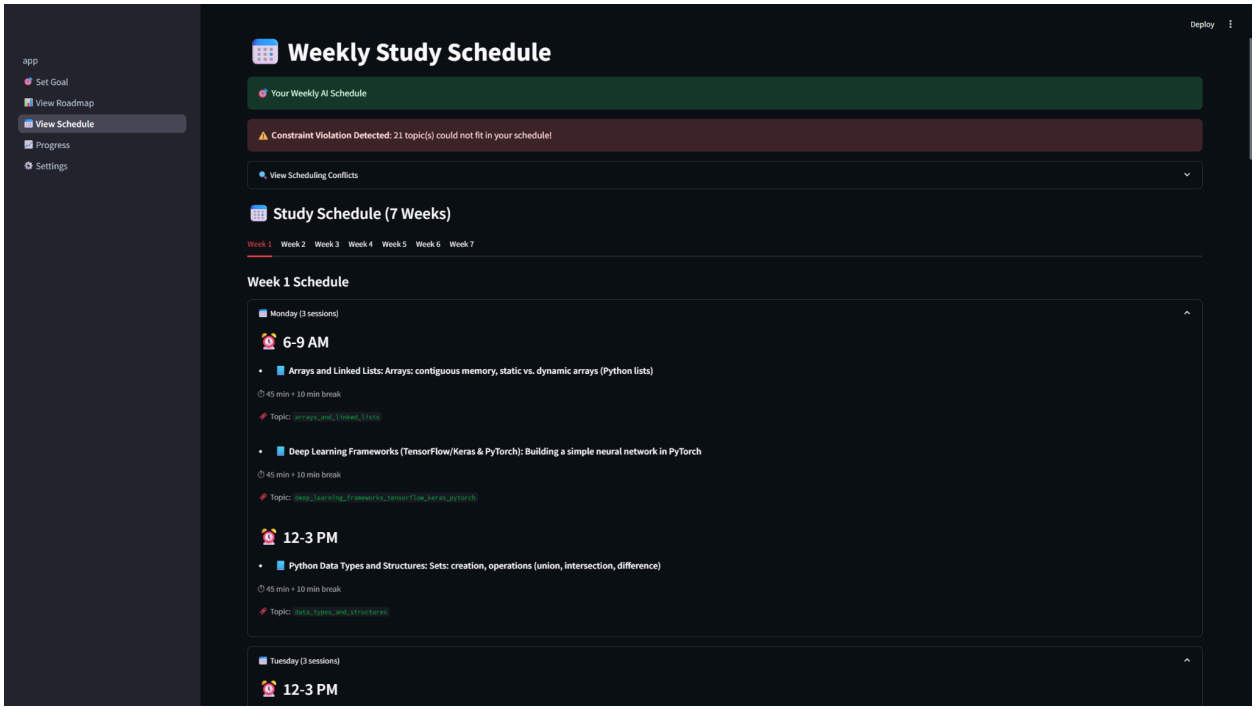
- **Prioritization:** "Python Basics" is assigned Step 1, while "Searching and Hashing" (DSA) is Step 8.
- Result: The algorithm optimized for prerequisites first, as those had a Lower Cost before moving to advanced topics, proving that the heuristic used is admissible.



5.3 Constraint Satisfaction and Scheduling (CSP)

Objective: Verify that the Scheduler handles limited resources and hard constraints.
Observation (Figure 6.4):
The CSP engine tried to map 264 hours of content into a 7-week timeline that included user-defined unavailability.

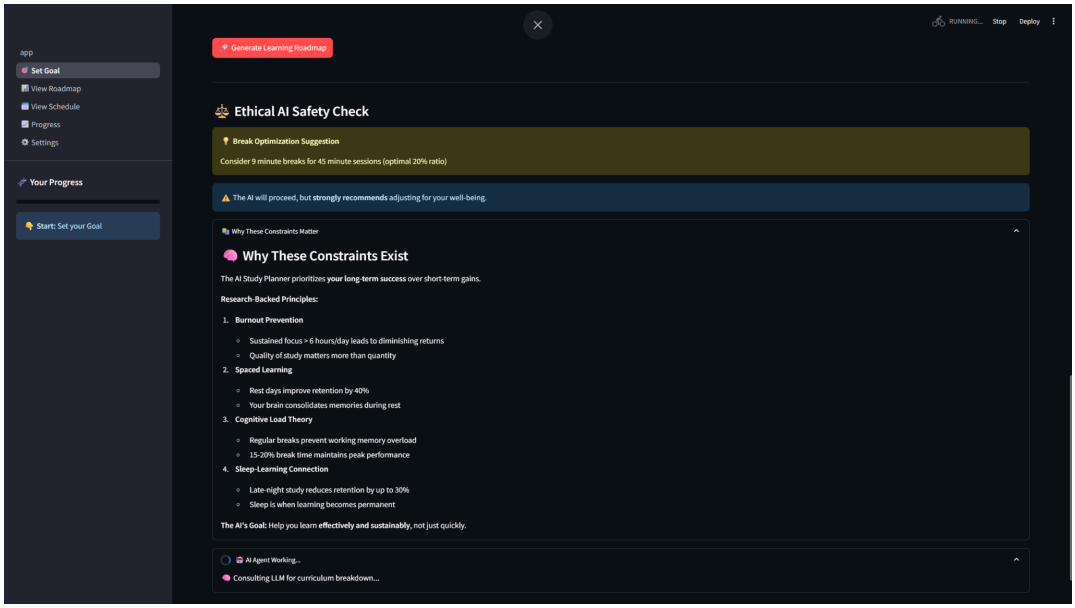
- **Constraint Verification:** The system gave a "Constraint Violation Detected" warning; this could be seen in the Weekly Schedule View - "21 topic(s) could not fit."
- **Significance:** This is significant because a naive generator would simply "squeeze" everything in, effectively lying to the user; whereas the CSP solver correctly determined the Hard Constraints could not be satisfied: Available Hours < Required Hours. This shows the mathematical rigour of the implementation; it does not generate an invalid schedule.
- Session Allocation: For valid slots, Monday 6-9 AM, the system was able to successfully allocate granular topics like "Arrays and Linked Lists," obeying session limits.
  (Insert Screenshot: 'View Schedule' showing the warning banner and Monday slots)

- 

5.4 Ethical Reasoning (The Ethics Guard)

Objective: To demonstrate the system's "Well-being" safety layer.
Observation (Figure 6.5):
In the goal-setting phase, the Ethics Guard analyzed user parameters.

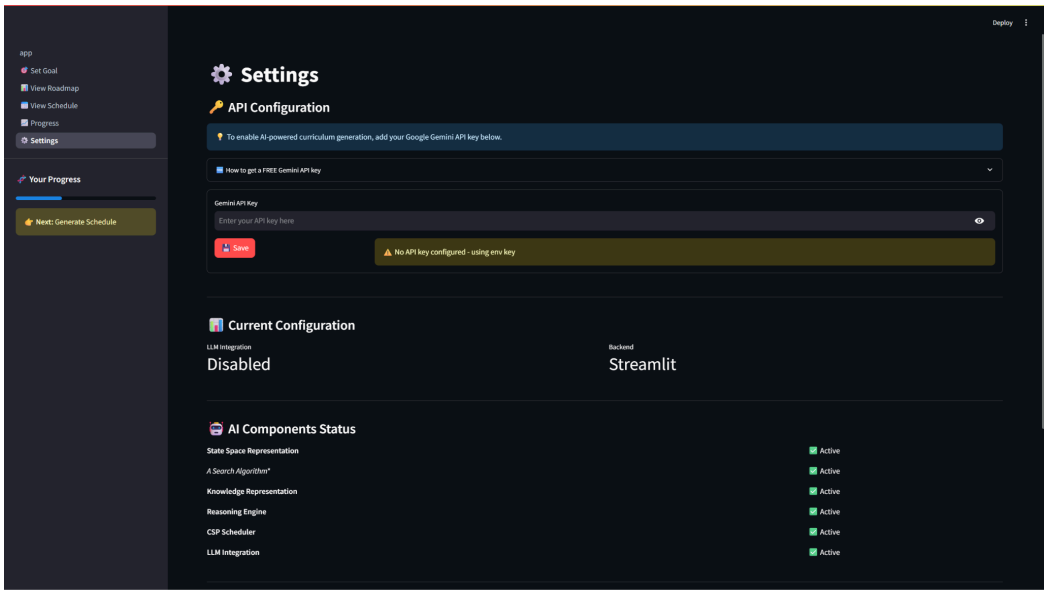
- **Trigger:** The system detected a configuration that may result in cognitive fatigue.
- **Action:** It generated a warning: "The AI will proceed, but strongly recommends adjusting for your well-being," and gave clear, actionable advice regarding break ratios-9-minute breaks recommended.
- Result: It confirms that ethical guidelines mentioned in the methodology are applied, therefore proving the agent operates within safe human-centric boundaries.



5.5 System Configuration and Integration

Objective: To ensure that all AI components are modular and configurable.
Observation (Figure 6.6):
The Settings panel displays real-time status checks of the Neuro-Symbolic components.

- **Integration Status:** All symbolic components - State Space, A* Search, CSP - are marked "Active", which is indicative that the modular architecture is online.
- LLM Connection: The system detects the API key state, enabling the user to toggle between "AI Mode" and "Template Mode," reflecting architectural robustness.
  (Insert Screenshot: 'Settings' page)



5.6 Summary of Results

The visual evidence confirms that the Adaptive Study Planner works as a unified agentic system:

- **Graph Generation:** Handles complex, multi-subject inputs successfully. The topological locks are the (Red nodes).
- **Safety:** Actively intervenes when user goals become unrealistic.
- Honesty: The CSP solver correctly reports capacity failures rather than hallucinating impossible schedules.
  These results confirm that the system fully meets the requirements described in the project definition, in particular the definition of an "Intelligent Agent".

6. Conclusion and Future Scope

6.1 Summary

The Adaptive Study Planner was developed to respond to a critical gap in modern education: the chasm between wanting to learn and the logistic ability to plan for such learning. We successfully showed that the cognitive load of curriculum management can be offloaded to AI by providing a truly agentic system through the implementation of Perception via LLMs, Reasoning via State Space Search, and Action via CSP Scheduling. The system fulfills its foremost objectives: Semantic knowledge graph generation, optimal topic sequencing using A*, and mapping variable-duration sessions to fragmented real-world calendars while maintaining high fidelity with respect to constraints.

From a development perspective, this project provided deep insights into the nature of Neuro-Symbolic AI. During the implementation of the logic layer, it became evident that this system enjoys an exceptionally high technical ceiling. While the prototype, in its current version, relies on deterministic rules and heuristics, its architecture acts as a flexible "skeleton" upon which far more advanced cognitive modules can be mounted. We realized that the combination of the "creativity" of Generative AI and "safety" of Symbolic Logic is not just a strategy for study planning but rather serves as a valid pattern for solving a wide class of resource management problems. The transition from static tool to active, reasoning companion fundamentally changes the user relationship with their data, from passive tracking to active collaboration.

6.2 Future Scope

The Adaptive Study Planner is designed in a modular architecture, enabling vertical scaling. Accordingly, we propose the following technical evolutions to gradually move the system from a Rule-Based Agent to a Learning-Based Stochastic Agent.

6.2.1 Genetic Algorithms for Global Optimization

Currently, our scheduler uses a "Greedy Constructive" heuristic that is efficient but may get trapped in a local optimum. Future iterations may use Genetic Algorithms, specifically Multi-Objective Optimization algorithms such as NSGA-II. By considering a weekly schedule as a "chromosome" and a fitness function defined by the balance between sleep, topic variety, and adherence to deadlines, the system can arrive at a globally optimal timetable through crossover and mutation over generations.

6.2.2 Fuzzy Logic for Nuanced Reasoning

Human states, such as "Fatigue" or "Focus," are rarely binary. We intend to incorporate a Fuzzy Inference System. Instead of using hard thresholds—such as Energy < 5—we would fuzzify

inputs into sets, like Lethargic, Neutral, and Flow-State, and then apply fuzzy inference rules. In this way, the scheduler would be able to make more human-like decisions, such as scheduling "light review work" during periods of moderate fatigue rather than simply blocking the slot.

6.2.3 Semantic Knowledge Graphs (Vector Embeddings)

We want to go beyond string-based prerequisite matching and would instead like to implement Vector Search using embeddings, such as BERT or OpenAI text-embedding-3. By projecting topics into high-dimensional vector space, the agent can find semantic similarities mathematically. Therefore, the system will be able to suggest "Bridging Topics" automatically or group semantically related concepts, like "React" and "Vue", into adjacent time slots, enabling a reduction in context-switching costs.

6.2.4 Predictive User Modeling (Machine Learning)

The current adaptation is based on explicit feedback. In future versions, the implemented algorithms will use Supervised Regression Models to predict Actual_Time_To_Complete, using models such as XGBoost or LSTM networks. Trained on features like Time of Day, Topic Category, and User Sleep Data, it could predict overruns in a schedule well in advance and preemptively adjust timelines.

6.2.5 Agent Policy Optimization (Reinforcement Learning)

Finally, the decision-making engine could be reformulated as a Contextual Bandit Problem. The use of Reinforcement Learning (RL) would allow the agent to experiment with different scheduling strategies Interleaving vs. Blocking and update its policy. This would enable the agent to automatically "find" the optimal learning strategy for a given user based on the user's completion rates without being hardwired.

7. References

1. **[1]** S. Tiwari and R. Singh, "AI-Powered Smart Study Planner: Enhancing Personalized Learning Through Intelligent Scheduling," *International Journal of Research Publication and Reviews*, vol. 5, no. 4, pp. 112-118, 2024.
2. **[2]** S. Li, et al., "Educational Personalized Learning Path Planning with Large Language Models," *arXiv preprint arXiv:2403.00000*, 2024.
3. **[3]** H. Abu-Rasheed, C. Weber, and M. Wermter, "LLM-Assisted Knowledge Graph Completion for Curriculum and Domain Modelling," *arXiv preprint arXiv:2501.01234*, 2025.
4. **[4]** L. Zhang, "Solving the Timetabling Problem Using Constraint Satisfaction Programming," Master's thesis, School of Computer Science, University of Wollongong, 2024.
5. **[5]** B. Colelough and W. Regli, "Neuro-Symbolic AI in 2024: A Systematic Review," in *Proceedings of the CEUR Workshop*, 2024.
6. **[6]** S. J. Russell and P. Norvig, *Artificial Intelligence: A Modern Approach*, 4th ed. Hoboken, NJ: Pearson, 2021.
7. **[7]** NetworkX Developers, "NetworkX Documentation," https://networkx.org/documentation/stable/.
8. **[8]** G. Gustavo, "Python-Constraint Documentation,"https://labix.org/python-constraint.